

**M A S A R Y K
U N I V E R S I T Y**

FACULTY OF INFORMATICS

**Analyzing trading strategies for
cryptocurrencies charts**

Bachelor's Thesis

ONDŘEJ BUČEK

Advisor: Bacem Mbarek, PhD

Department of Computer Systems and Communications

Brno, Spring 2022



Declaration

Hereby I declare that this paper is my original authorial work, which I have worked out on my own. All sources, references, and literature used or excerpted during elaboration of this work are properly cited and listed in complete reference to the due source.

Ondřej Buček

Advisor: Bacem Mbarek, PhD

Acknowledgements

First of all, I would like to thank Bacem Mbarek for guiding me through this whole process. I know it must have been hard at some moments, and I am really grateful for your patience with me. Many thanks belong to my family for providing me with a warm, welcoming home and supporting me on in studies. You allowed me to focus on the things I wanted and needed. Furthermore, I would like to thank my amazing friends that helped me go through my studies and be supportive during the time.

Abstract

This bachelor thesis describes the problems that a programmer of automatic trading systems in the environment of cryptocurrencies may encounter. These problems may be the design of a strategy that takes into account a specific market, a currency pair, and technical capabilities of specific technologies and market exchanges.

The thesis describes some strategies, including the publicly known models Martingale and Turtle strategy, on which certain problems associated with automatic trading can be described.

The result of these analyzes and the result of the work is a created program that allows traders to effectively backtest strategies and thus simplify the work of traders.

Keywords

trading strategy backtesting, backtesting program, martingale analyzes, turtle strategy analyzes

Contents

1	Introduction	1
2	Terminology	3
2.1	Trade orders	3
2.2	Fees	8
3	Designing strategies and backtesting software	9
3.1	Designing strategies	9
3.1.1	Negatives of algorithmization	9
3.1.2	Positives of algorithmization	10
3.2	Backtesting and trading programs	10
3.2.1	Metatrader	10
3.2.2	NinjaTrader	11
3.2.3	TradingView	12
3.2.4	3Commas	12
3.2.5	TrendSpider	13
4	Algorithmic description of the strategy	14
4.1	Indicators	15
4.1.1	Moving Average	16
4.1.2	RSI	16
4.1.3	Algorithmic strategy description	16
4.2	Martingale	17
4.2.1	Algorithmic strategy description	17
4.3	Constant value of an asset at Sell and Buy levels	20
4.3.1	Algorithmic strategy description	21
4.4	Price action	24
4.4.1	Algorithmic strategy description	24
4.5	Turtle strategy	25
4.5.1	Algorithmic strategy description	25
5	Backtesting program	28
5.1	Strategy analysis	28
5.2	MA RSI strategy	29
5.2.1	Result analysis	29
5.3	Price action strategy	31

5.3.1	Result Analysis	32
5.4	Turtle strategy	32
5.4.1	System 1	34
5.4.2	Analyzing System 1 Results	34
5.4.3	System 2	34
5.4.4	Analyzing System 2 Results	34
5.5	Evaluation of backtesting strategies	36
6	Conclusion	37
	Bibliography	38

1 Introduction

Cryptocurrency trading has become more and more popular in recent years. The vision of getting rich quickly attracts more and more people who want to use this tool. Unfortunately, due to poorly set strategies and insufficient education from both a fundamental and trader point of view, many people end up in negative numbers.

Emotions play a significant role in these failures. Failure to manage personal discipline when a person sees large negative or even large positive numbers in the account often leads to bad decisions. Therefore, many people are looking for a tool to filter out their emotions and prove to be profitable. Various robots and automated scripts that trade according to predefined rules can be used. But if programming such successful robots was easy, everyone would do it, and everyone would be successful.

I have been operating in the crypt market since 2017 and since 2020 I have been actively involved in communicating with other traders to algorithmize some strategies. Over the years, I have created countless scripts to understand the market better and help to analyze it. A large part of them is dead ends, on which, however, it was possible to build own program that allows me to trade specific strategies. The program at the current stage is only used to test strategies in the real market and is definitely not intended for commercial use. An integral part, however, was the creation of a backtesting program that can analyze certain strategies and obtain outputs on which to build further analysis and evaluation of the strategy.

Throughout this issue, it is important to know the problems that may arise when creating and analyzing strategies, ideally with their solutions. The next chapters will be used for this. At the same time, there are a lot of opportunities to think about creating a strategy, but it is always necessary to adhere to certain fundamental know-how, rules, and restrictions on which the strategy can then be built. One such rule is the general investor lesson: "Past profits do not guarantee future profits." In other words, no matter how promising our strategy is, we are still moving in the field of statistics and therefore it is necessary to take into account different scenarios. Furthermore, I will discuss what problems may be encountered in designing different strategies, I will

describe the possibilities of backtesting, a few existing platforms that allow backtesting, and evaluate the results of a few model strategies from the newly created program, which was the aim of this work.

2 Terminology

Let's mention a few similar terms that I will encounter on the following pages

- **Price Chart** - Shows historical price movements and it is a basic tool used to analyze strategies. There are several different types of views.
- **Time frame (TF)** - The graph is divided into equal length intervals, where each interval is somehow specifically represented on the graph. We can most often meet with the designations 1W (1 week), 1D (1 day), 4H (4 hours), 1H (1 hour), 5m (5 minutes), 1m (1 minute).
- **Open price (OP)** - Value at the beginning of a specific interval.
- **Close price (CL)** - Value at the end of a specific interval.
- **High** - Highest value during the interval.
- **Low** - Lowest value during the interval.
- **Candlestick chart** - Price chart, where for each period we record the values OP, CL, HIGH, LOW, where the area between OP and CL is rectangularly colored with 2 colors according to the type of candle (decreasing, where OP is greater CL, is often colored red and ascending, where OP is smaller than CL, is often colored green). See image 2.1.
- **Price action (PA)** - A method that analyzes the graph based on price movement

2.1 Trade orders

In order to execute the trades themselves and trade, I need to have specific orders to execute the trade. In this terminology section I will look at the types of orders and their division. The basic division is according to the direction of trade, whether people buy (the designation Long or BUY) or people sell (the designation Short or SELL).

It is possible to handle and enter different types of these orders on exchanges. I will now talk about following orders:

- **Market**
- **Limit**

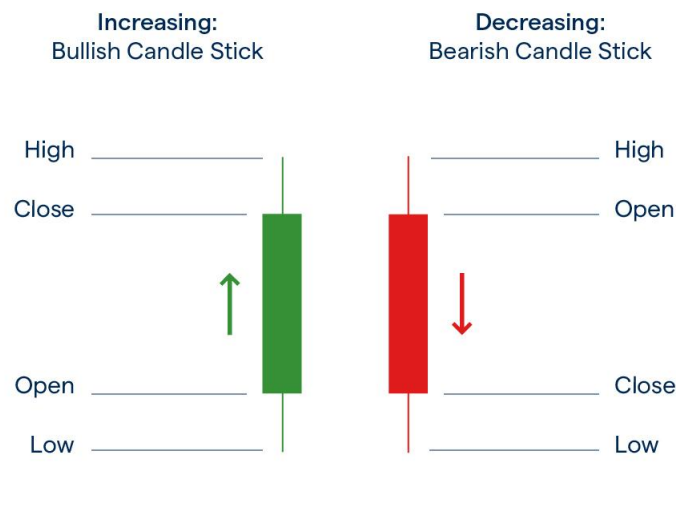


Figure 2.1: Example candles [2]

- **Limit Stop**
- **Trailing Market**
- **Trailing stop**
- **Take profit**

Let's show the different types of orders in a simplified example with the apple market. Imagination of a market in the square. The square is owned by the city and the city can regulate the shops in the market.

The market will look like this: Let's have a sign with the current price of apples, as well as a number of stalls built next to each other. Stalls are divided into two parts. On the left, the stalls that buy apples lower than the current price will be sorted in ascending order, and on the right, the stalls that sell apples at a higher price than the current price will be sorted in descending order. In the middle is a stall that offers or sells apples at the current price.

Next, let's have customers coming to market. They do not only buy apples, but they can also have their own apples for sale. We divide these customers into two parts: Those who have a specific price at which they want to sell or buy will go to the stand with the given price

or if the stand does not exist, he gets his own stand with the given price and joins rows of stalls. Those who do not have a specific price come in the queue style system: "I do not care what the price will be when buying, I just want apples". Gradually the market owner takes these individual customers with the following algorithm.

- If the customer buys apples and the stand with the current price sells and the stand offers more apples than the customer requires, then the customer buys apples at the current price and leaves. If there is a shortage of them at the stand, the customer buys the stand and moves 1 right to the stand with the lowest price of his apples. This will change the price on the board with the current price of apples.
- If the customer buys apples and the stand with the current price also buys, the customer, as in the previous case, moves 1 to the right and the price on the board changes.
- If the customer sells apples and the stand with the current price buys and the stand buys more apples than the customer offers, then the customer sells the apples at the current price and leaves. If the customer has more than the stand buys, the customer will move 1 left to the stand with the highest purchase price of apples after the demand has been met. This will change the price on the board with the current price of apples.
- If the customer sells apples and the stand with the current price also sells, then the customer, as in the previous case, moves 1 left, and the price on the board changes.

What does this have to do with the stock market and orders? In this model, the market manager represents the stock exchange, apples a specific traded object (commodity, shares, EUR, BTC, etc.) customers standing in the queue **Market orders** and stands **Limit orders**.

Market orders

This is how in general orders are called that say: I want to buy or sell (I want to stand in line at the market) and I do not care what the price will be when it is my turn in the queue to settle. I will call this queue

the Order Queue. Note that in the apple market, trading takes place in significantly higher time units than in the automated stock exchange system.

Limit orders

This is how we refer to orders that create an offer to trade. "I want to sell or buy apples at a specific price. Or, when the price reaches this price, I will be ready with the apples." It creates liquidity in the market

Stop market

This type of order is sometimes referred to as a Stop-loss. Let's see how such an order works.

There may be another number of customers in our market who are standing nearby and looking at the price list. Those who want to buy apples have their price (Stop price) set, which is HIGHER than the current one, and when a price that is equal to or higher appears on the board, it is queued for **market orders**. Those who want to sell apples have their price set, which is LOWER than the current one, and when a price that is equal to or higher appears on the board, they are placed in the queue of **market orders**.

Since there may already be a lot of customers in the queue who may have large volumes of apples they want to trade, it is possible that the price of apples will shift significantly to the detriment of our Stop market customer.

In practice, Stop-loss orders are used to cover losses or cover profits in the event of an unfavorable market situation.

These 3 so far are the most basic, but I will mention a few more that are possible to meet.

Stop limit

This is an extended Stop market. The extension is that in addition to the Stop price, these customers also have a Stop limit price, which says what the lowest or highest price the customer is willing to buy or sell when it's their turn in the queue, and the price has since risen. What came in the queue significantly shifted.

Trailing stop

Again, this is an expanded Stop market. We extend by the percentage deviation from the highest or lowest peak that the price in the graph created when we want to sell or buy. A group of customers who want to sell or buy apples will appear on our market. Each has its own percentage deviation x and monitors the development of the price. In the beginning, their Stop market calculated according to the deviation is automatically set for them. If the price rises or falls, then this Stop market is recalculated according to x and the current price of apples. If the price breaks their Stop-market price, then, like the Stop market, customers will be queued.

It is used in practice when, for example, a strategy have a hypothesis where it knoww that the individual peaks and corrections in the trend are in the range of x percent. And if the correction is bigger, then the trend is probably reversing to our detriment, and therefore it can save profits and suppress significant losses.

Leveraged position

Leverage also has an important role in the evaluation. I will explain what this means. There are different types of accounts.

For example, a SPOT account allows traders to trade only physical assets (such as different amounts of currency) that they have "physically" in the account.

MARGIN-type accounts extend this possibility to traders by significantly increasing tradable volumes. Leverage trading (or margin trading) is basically borrowed money trading. When opening a position, the trader makes a small deposit (margin), for example 10% of the entire order, and the rest of the order is financed by the broker. If the trade is profitable, the trader's margin and return will return. Otherwise, the margin will cover the incurred losses. If the loss is greater than the margin, the broker automatically closes the trade. This situation is called **Margin call**. It is possible to use the entire account as a margin, then it depends on the specific ratio that the stock exchange or broker is willing to provide.

2.2 Fees

An important factor in creating strategies are also the types of fees and their size, which are associated with the strategy.

Spread

One of the basic types of fees is spread. Specifies the difference between the current sale and purchase price. High spreads can then significantly affect fast strategies that operate at low timeframes.

Volume charges

Typically, a charge on the volume of traded currency appears on cryptocurrency exchanges. This fee may vary from account to account type and can be around 0.01 – 0.1%, however, as a result it works as a larger form of spread. This significantly increases the success that a trader must make in order to be profitable. Some exchanges distinguish between Maker and Taker fee. Maker fee is for those who create liquidity, or orders that come to the order book in the form of limit orders and Taker fee for those who withdraw this liquidity, or market orders (for example, the Stop-loss market is included). On some exchanges (eg Bitmex [4]) there is a positive fee when trading maker orders. With this, I can close the basic terminological passages of this work and move on to concrete designs of backtesting softwares and strategies.

3 Designing strategies and backtesting software

Traders have many different software on the Internet that can help them fine-tune the strategy and automate it. I will take a look at a few of them and say something about designing strategies and backtesting.

3.1 Designing strategies

In order to be able to choose the right technologies that we will need in trading, automated bots programming and backtesting, we must have a draft strategy.

The proposal needs to specify a specific market or markets (stocks, currency pairs, commodities, etc.), fundamental (whether we trade according to world reports, price movement analysis, indicators, etc.) and an algorithmic description of the strategy (individual conditions that create signal to enter, exit, or adjust the position size) and then select specific technologies and exchanges that will help us to do the trading itself.

There are countless markets in which to trade, however, in this work, as I mentioned in the introduction, I will deal exclusively with the trading of cryptocurrencies.

The approach to this issue is different compared to human trading [8] and otherwise if the succesful trading system is creating. This work is interested in the second way in which I must specify both the positives and negatives.

3.1.1 Negatives of algorithmization

The negatives depend on the experience of the programmer. The basic problem is the lack of experience and intelligence in pattern recognition, as the robot works on the basis of conditions and mathematical models. If the programmer is sufficiently experienced, he can also use AI, which is usually a tool that can significantly help, for example, in creating trend lines that stand as a building block in many strategies.

Another shortcoming lies, for example, in the ability to analyze sudden changes in the market and gradual, in the longer term, fundamental changes in market behavior.

3.1.2 Positives of algorithmization

Absence of emotions. As I mentioned in the introduction, many traders do not make money trading because they cannot control their emotions, let themselves be controlled by them and start making trading mistakes. The specific problems are fear of losing money and greed. As a result, this manifests itself in non-compliance with the trading plan - premature closing of trades, incorrectly set stoplosses, excessive trading even in moments when the market does not give sufficient input signals.

One of the other positive features is the speed of data analysis. Thanks to automation, we are able to make immediate decisions, which can be very important for the profitability of the strategy.

Furthermore, of course, we can combine the positives of the trader and the robot, such as manually created indicators (trend lines) and constant control of the graph by a robot, which creates a trade at the moment for example when the graph touches this trendline.

3.2 Backtesting and trading programs

This chapter is about few current options that can be used for analysis, backtesting and possible algorithmic trading. Each individual software can be divided according to whether they handle one or more of these functionalities. As far as trading opportunities are concerned, it is very important what market is it supposed to operate in. There are some options at the crypt market and others at FOREX, for example.

3.2.1 Metatrader

It is a platform that allows people to analyze charts, make orders and connect to Metatrader, as well as other similar programs, after programming with the account of a specific broker, it is possible to program own indicators and strategies. Currently 2 versions are used. Metatrader 4 and Metatrader 5 (MT5). The languages mql4 [23] and mql5 [24] are used for this.

These languages are built on C++. The MetaEditor editor is built into the programming itself, where after creating the script, one can

insert it as an indicator or trading script. For the automatic trading program itself, Metatrader needs to be opened and the script running.

From a personal point of view, I wouldn't recommend using this way very much. Setting up the user interface, shortcuts and overall ease of working in this editor was clearly worse for me compared to editors such as Pycharm or JetBrains.

In the appendix [9] I enclose one of my programs that prevented automated trading at specific times during the year. The source code, especially in some files, certainly cannot be described as well programmed. From the point of view of laboriousness, optimization of programming performance and requirements for the resulting system, it was easier to create larger files than nicely algorithmized functions due to the inconveniences of debugging in MetaEditor.

One of the positive features of MT5 that I use is the Python library, which allows me to connect to my accounts. [22]. As a result, it is possible to connect to only one account, which prevents multiaccounting on one device. However, this connection through Python opens up countless possibilities for connecting to other software due to the simplicity and public knowledge of this language.

3.2.2 NinjaTrader

This platform, like Metatrader, allows you to analyze charts and create various trading scripts and indicators that help trading. The language is called NinjaScript and is built on C#. The platform works on pricing, where the free version allows basic analysis and testing of strategies on the chart, and paid ones allow you to connect to a few specific brokers, do trades and create various alerts according to the signals from the charts.

I don't have much personal experience with this platform by myself, because for specific projects it was necessary to link the program directly to a specific cryptobourse or work with Metatrader, and I created another form of strategy analysis in Trading View, which I will mention.

3.2.3 TradingView

It is the largest online platform and social network that connects data from a large number of world stock exchanges and institutions. What does Trading View itself say about itself?

"What's TradingView? We're a charting platform and social network used by 30M + traders and investors worldwide to spot opportunities across global markets." [20]

This position on the world stage also won thanks to easy access through the web interface [21]. The disadvantages of the free version are, for example, the limited number of indicators that can be inserted into the chart, limited chart sizes, the number of alerts that can be sent to email, telegram, signal, etc. The list is written on the following website [26]. However, this tool is very suitable for basic analysis.

You can also program your own indicators and strategies. A language Pine script [29] is used for this. It is not based on any other language, however it has similarities to Python or Kotlin in a few ways. The language itself is easy to learn, however, one of the problems that is similar to general analysis programming is understanding how candles are recorded in the data.

For example, if I analyze the historical performance of a strategy that analyzes a graph when closing or opening a candle, there is a big difference between whether the function that returns the *n*th last candle sends this data at the time the current candle is closed or opened. Following the strategy, it may happen that the strategy may include data from the current candle, which will happen in the future compared to the moment when a particular moment is analyzed. Learning to program in this language is therefore quite an understanding of the mechanics of storing and using data of individual built-in functions. At that moment is great to have hands over the dataset and be able to deeply control each trading signal.

3.2.4 3Commas

This web application [5] allows people to link and trade on different accounts across a single platform. 3Commas has the option of setting up its own trading robot. It does not work as in the above systems.

3Commas has robots based on a specific foundation (e.g. DCA strategy) and the trader only sets various constants and parameters that will specify his strategy.

3Commas also allows connection to other applications such as Telegram and Signal. The API connection with 3Commas enables easier trading on multiple accounts compared to other already mentioned platforms.

3.2.5 TrendSpider

This platform [27] has set up pricing for 7 days for free and then a few tens of dollars/month. It allows you to create strategies using a simple design environment where the trader clicks on the individual conditions. The advantage is e.g. the use of automated trendlines and supports which other softwares can't do and it can simplify working with the chart.

Conclusion

The problem for which I could not find an existing solution was to test various own strategies on various timeframes for free and to have a clear overview of the backtest, the performance of the strategy and the individual results. At the same time, I have the concept of a backtesting program, where I can copy the core and link it to other of my already created projects, such as a new application that examines the real trading of a specific strategy and the performance of connections with individual crypto bourses. At the end of the thesis, I will show and describe this backtesting program.

4 Algorithmic description of the strategy

In this chapter I will talk about a few trading principles and specific strategies, some of which I will backtest in the next chapter.

First of all, it is necessary to specify what the robot should actually trade. It may be wanted to build a system that analyzes data from a chart, calculates various indicators, collects data from multiple exchanges and locations, which it further analyzes, or even analyzes text and information from world portals using text recognition.

Each of these proposals will affect the specific technologies we use as a result. At the same time, when analyzing low-timeframe graphs (1M = 1 minute, 3M = 3 minutes), the speed with which we will create any final orders will most likely be decisive. There are a few general lessons to be learned from working with strategies.

Past profits do not guarantee future profits

It has been already in the introduction. This is one of the basic general investor lessons. The idea is that while the strategy can be very effective on historical data, we always move on the basis of the probability that the principles may change.

Some principles may be more reliable (e.g. long-term economic growth compared to times of crisis) than others ("Bitcoin is constantly rising in the medium term." Cryptocurrency regulation, for example, is one of the current issues where the question is what it is or it will be going with the cryptocurrency world)

Correlation does not imply causality

The fact that two phenomena are similar does not mean that there must be a relationship between these phenomena that determines this similarity in the future. This must be kept in mind, especially in the more detailed specification of creating the core strategy, inputs and outputs to trades and subsequent backtesting.

For example, if a trader optimize the constants for indicators WITHOUT fundamental knowledge of why he makes the changes and just try to improve performance on historical data, it will only lead him to

have a successful strategy on historical data, but it certainly does not imply that this strategy will have a higher success rate in the future.

If I say that the range of the maximum drawdown that Bitcoin achieves has a declining tendency, without knowing the reasons behind it and building a strategy on this thesis, then the success of such a strategy cannot be relied upon much.

Capital protection

If trader do not generally trade and invest in the longer term and do not have the opportunity to "sit down", or wait for our cyclical market to return to neutral numbers, it is necessary to determine the moment when he accepts the loss, because there is a sufficient probability that the market will go against him. Stop-losses help to do this. A fixed rule that helps us minimize the effect of such negative events. At the same time, there is the amount of money we are willing to risk for one such trade. Different sources make different recommendations, however, it is always better to start at small percentages and per mille and increase than vice versa.

The following strategies have a fundamental basis in mathematical models, price movement of the chart and indicators that add meaning to these charts.

4.1 Indicators

Trading indicators are mathematical calculations that are displayed as lines on a price chart and can help traders identify specific signals and market trends [13].

Indicators are various models that, based on specific information, bring additional value for graph evaluation. A large number of indicators can be found on the Internet and in the literature. By a suitable combination, more accurate signals for inputs and outputs can be obtained. I will further develop 2 indicators that will help me to build a strategy, which I will test later.

- Moving average (MA)
- Relative strenght index (RSI)

4.1.1 Moving Average

A moving average [15] is an indicator that is used to make it easier to detect a time series trend. The base type is calculated as the average of n consecutive periods.

The most well-known basic type of MA is **SMA** - Simple moving average (which takes a simple averaging of the values of these periods, as mentioned above) and another used is **EMA** - exponential moving average (takes individual periods, where it gives weight to the period closer to the present).

$$SMA = \frac{A_1 + A_2 \dots + A_n}{n}$$

$$EMA = (Value_today * \frac{smoothing}{1 + n}) + EMA_last * (1 - \frac{smoothing}{1 + n})$$

4.1.2 RSI

RSI [16] measures the internal strength of an asset. This indicator belongs to the group of oscillators. The predefined range for this indicator is 0-100, in which the indicator moves. The formula is used to calculate RSI:

$$RSI_t(n) = 100 - [100 / (1 + U(n) / D(n))]$$

U (n) - average of net growth closing changes over the period

D (n) - average of net decline closing changes over the period

4.1.3 Algorithmic strategy description

Let's place 3 indicators RSI (40), MA (12) and MA (50) in the graph. Furthermore, for each candle determine the max_OP and min_OP, as the highest and lowest OP of the last 20 candles. The robot will behave as following:

- If a trade is open, the robot waits for the faster MA (12) to cross the slower MA (50) and then close the trade.

- LONG open: The robot waits for an RSI less than 50 and the MAs intersect. SL is set to min_OP last 20 candles. The risk per trade is 1%.
- SHORT open: The robot waits for an RSI less than 50 and the MA intersects. SL is set to min_OP last 20 candles. The risk per trade is 1%.

Results of the strategy are described below 5.2.

4.2 Martingale

This strategy has its origins in mathematical models that would increase the chances of winning in gambling. The strategy works on the basis of doubling the bet after an unsuccessful attempt. I bet 100 and I lose, so I bet 200. If I lose again, I double my bet to 400, then to 800, and so on. [18]

There are many variations of this principle, such as adjusting the growth function, which indicates how the next bet should increase. Let's look at the issue in the following draft strategy.

4.2.1 Algorithmic strategy description

- If no trade is open, the robot enters the trade. The direction trades is reversed than the previous closed trade (Default set to LONG). We will call this original level OP.
- Take profit (TP1) is fixed at 1% from entry in the direction of trade and the level in the opposite direction, which we call sl-trigger, is also fixed at 1% from entry.
- The moment we get to the sl-trigger level, we open the trade in the opposite direction with TP2 level 1 % from this sl-trigger point (It is possible to have both LONG and SHORT trade open at the same time). Lets call this level, where we have now opened the store, as OP2. Gradually, trades at OP1 and OP2 levels will open up to us. So now we have 3 levels: TP, OP1=sl-trigger2, sl-trigger=OP2, TP2.

4. ALGORITHMIC DESCRIPTION OF THE STRATEGY

- A series of such trades will be closed only at levels TP1 and TP2.
- The result of a series of martingale trades leads to a profit of 0.5%, which in our view means that we enter the first trade with

$$\frac{deposit * 0.005}{0.01} = 0.5 * deposit$$

half of the entire account.

- As long as we have these stores open, every time we get to the level of OP1 resp. OP2, we will add a new trade to the given direction so that upon the subsequent achievement of TP1 resp. TP2, we were in profit of 0.5%. The equations must be met:

$$\begin{aligned} & |(\frac{TP1 - OP1}{OP1} * OP1_position - \frac{TP1 - OP2}{OP2} * OP2_position)| \\ & = \frac{0,5}{100} * deposit \end{aligned}$$

analogously for OP2, where OP1_position and OP2_position are the sizes of open trades on the given levels and deposit is the original size of the account.

Let's see a model example 4.1: In the table ?? we see how in this model example we gradually change the sizes of trades for deposit 2000 and profit per cycle 0.5%, if the graph oscillates at values between TP1 and TP2 without them touched. Each line records one of the points of the sequence OP1, OP2, OP1, OP2, TP2.

At the beginning we have a LONG of size 1000, then we add SHORT 2970, we raise LONG to 7000, we raise SHORT to 14850 and then we break through TP2, where we earn 10. The remaining parts of each line tell us how much we would get at a given price level at that moment, if we did not make any further increases.

It can be seen that the total volumes we give to this trade and the potential losses are rising very quickly, if we stop adding other

4. ALGORITHMIC DESCRIPTION OF THE STRATEGY

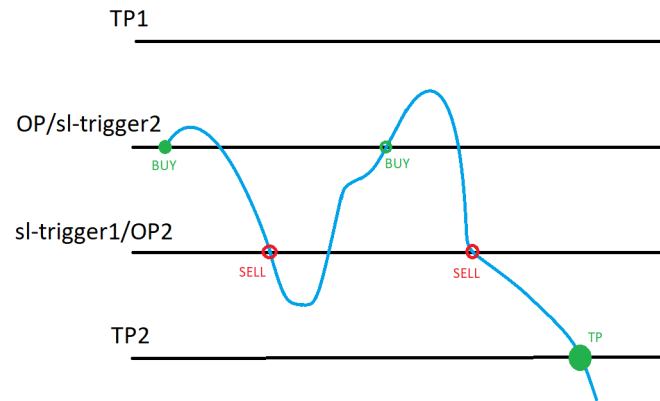


Figure 4.1: Example of Martingale movement OP1, OP2, OP1, OP2, TP2

Table 4.1: Changing the size of positions in the sequence OP1, OP2, OP1, OP2, TP2

Dif	0,01			
Deposit	2000			
Risk	0,005			
Prices	980	990	1000	1010
		Short size	Long size	
OP1	-20	-10	1000	10
OP2	10	2970	-30	-50
OP1	-110	-70	7000	10
OP2	10	14850	-150	-230
TP2	10			

positions after a certain level, for example, for technical reasons. At the moment when we add the fees from the volume (e.g. 0.1%), because we have to execute all existing trades in the Market or Stop-market type, so we can get to the moment when the stock exchange no longer allows us to open trades and in the worst case we get a margin-call. In this case, we will lose our entire account.

On the one hand, of course, this is something that no trader wants, but in order to be able to evaluate the strategy well, it is important to first look at it in the long run, whether the gradual earning of 1% can earn enough so that if it constantly makes profits into a safe non-trading account and we "delete" the account with a margin call, it still will be in the red.

The question is whether such a model, which we have proposed, will lead the robot to do so. It could achieve this if the market was volatile enough and always got out of our 3% range very quickly and made a profit. Such a model, when we start trading at any moment, is harder to backtest, because it can realistically happen that if we take any moment and analyze all of them, there will be successful profit moments and highly loss moments.

As a result, a fundamental basis is important for this mathematical principle, which will not change in the future, because through the exponential growth of volumes in trade, every small change can have a significant effect. The difficulty and issue of backtesting can be demonstrated on this strategy. My backtest program is not designed for this strategy either, so I will not include this strategy in the final chapter of backtesting.

4.3 Constant value of an asset at Sell and Buy levels

Most people who move on the crypto market own some BTCs, use some for long-term holding and trade some. One of the basic rules of trading and investing in general is to buy low and sell high.

Let's look at a simple mathematical principle that uses DCA (dollar-cost averaging) methods. This principle says that the value of BTC in USD that we hold is still constant [14].

I consider a model where BTC costs 50,000 USD and I currently hold 1 BTC. The value of BTC that I will maintain (let's denote c) will

therefore be $1 * 50,000 = 50,000$ USD. At the same time, I will have a certain amount of USD aside (how much, we will explain this later) so that BTC can be bought back.

Let's specify the constant k price level difference at 10 %. So there are sequences 4.2:

Table 4.2: Price levels of mathematical strategy

Rising Price Levels	Falling Price Levels
50000	50000
55000	45454,5
60500	41322,3
66550	37565,7
73205	34150,7
80525.5	31046.1
88578,0	28223,7
97435.9	25657.9
107179.4	23325.4
...	...

4.3.1 Algorithmic strategy description

- As it grows, I will behave as follows. I will sell enough BTC to get the rest of the BTC I hold to the original 50,000 USD. For example if the differences in price levels are $1/10$, then when it moves to a higher level, I sell $\frac{k}{1+k} = \frac{0.1}{1.1} = 1/11$ the amount of BTC I own.

In the first step from 50,000 to 55,000, this means selling a BTC price of $1/11 \text{ BTC} = 5,000 \text{ USD}$. There will be 0.909090 BTC left after the trade. In Table 4.3 is possible to see the changes in the amount of BTC if the market would only grow.

Table 4.3: Changes at 100 % growth in Math strategy

Price 1 BTC	BTC Balance	Total earned USD
50000	1	5000
55000	0.909090909	10000
60500	0.826446281	15000
66550	0.751314801	20000
73205	0.683013455	25000
80525.5	0.620921323	30000
88578,0	0,56447393	35000
97435,9	0,513158118	40000
107179.4	0.46650738	45000
...	...	...

- When declining, I will behave as follows. I will buy enough BTC to make 50,000 USD with the amount of BTC I hold. For example if the differences in price levels are 1/10, then when moving to a lower level I buy $k = 1/10$ the amount of BTC. In the first step from 50,000 to 45454, this means a purchase of $1/10$ BTC = 4545 USD. Then I will hold 1.1 BTC. In Table 4.4 I can see the changes in the amount of BTC if the market just fell.

Table 4.4: Changes at 100 % drop in Math strategy

Price 1 BTC	BTC Balance	Total Expenses USD
50000	1	4545,454545
45454,54545	1,1	9090,909091
41322,31405	1,21	13636,36364
37565,74005	1,331	18181,81818
34150,67277	1,4641	22727,27273
31046,06615	1,61051	27272,72727
28223,6965	1,771561	31818,18182
25657,90591	1,9487171	36363,63636
23325,36901	2,14358881	40909,09091
...	...	...

Of course, the market is not just growing or declining, but it is rippling (consolidating), sometimes a decline, sometimes an increase and sometimes the price stagnates. So when is the profit itself realized? Let's see what happens when the price gets one level higher and then one level lower back to the original.

After the first step, I keep BTC from the original 10/11 Bitcoin and 5,000 USD more than the original. When I go back to the price level of 50,000, I buy $1/10 * 10/11$ BTC for $1/11 * 50,000 = 4545$ USD. So thanks to this move, I will have 1 BTC and $5000 - 4545 = 454$ USD extra. I will achieve the same result if the market first falls down and then returns.

For a specific k , there is a formula that says how much it would earn on such a wavy motion.

$$gain = c * k * \frac{k}{1+k} = \frac{c * k^2}{1+k}$$

Every strategy has its pitfalls. The danger of this method lies in the amount of USD that needs to be kept aside to be used for buying later. The moment the decline of x tens of percent comes, it may not be possible to use the strategy, because none USD left.

This means that the fundamental premise of this strategy is the thesis that BTC is constantly growing in the long run and in the short to medium term the decline is a maximum of x percent. If we accept the cyclicity of the BTC market, then in the last 5 years the maximum decline was max 85 %, which for $k = 0.1$ says that the traders would have initially hold 1.5 times the BTC in USD for the strategy to work constantly. At that moment, the amount of money N in one move up and down, generates an appreciation of $\frac{N}{1+1.5} * \frac{k^2}{1+k}$.

Strategy conclusion:

If someone takes the assumption of cyclicity and continuous long-term growth, which is the basis of the above strategy, the choice of HODL strategy in the middle of the cycle can be buy and sell at specific parameters such as FEAR & GREED index, RSI and other reports and indicators indicating an unhealthy market, significantly more profitable in the long run.

This mathematical strategy can potentially be used as an adjunct to periods of cooling and recession, with BTC heading sideways. The

problem is determining when to move from one strategy to another. Here we are already at high timeframes and more on the fundamentals and long-term market situation than in the area of pure backtesting values on the chart.

As in the Martingale strategy, I have shown the issue of backtesting certain types of strategies. The robot is not designed for this strategy either, due to the uncertainty of the entry level settings.

4.4 Price action

I will create this strategy based on the theory that strong support will come when achieving support or resistance if the reflection is to be successful. We can use swing theory and candle patterns for this. Swing low is a red candle formation followed by green and Swing high is a green candle followed by red. Often the price between these two candles can represent a support and resistance levels. There are two candle patterns I will use possible to see on image 4.2.

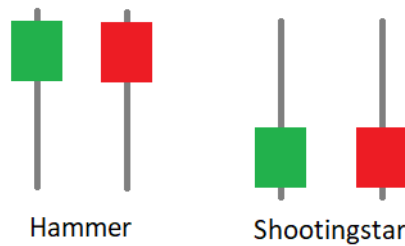


Figure 4.2: Candle patterns Hammer and Shootingstar

4.4.1 Algorithmic strategy description

- Enter Long with the Market order if the last swing low (the last formation of a red candle followed by a green one) contains a Hammer-type candle, where:

$$(\min(OP, CL) - LOW) / (HIGH - LOW) \geq 0,7$$

- For Long is SL set at the LOW of this swing low and TP at the same distance from the OP as the SL.
- Enter Short with the Market order if a swing high has been created (green candle formation followed by red), which contains a Shooting Star type candle, where

$$(HIGH - \max(OP, CL)) / (HIGH - LOW) \geq 0.7$$

- For Short is SL set at HIGH of this swing high and TP at the same distance from OP as SL.
- The risk to trade is 1%.

Results of the strategy are described below 5.3.

4.5 Turtle strategy

This strategy dates back to 1983. Richard Dennis became a successful trader at the time. He made a huge fortune and often talked to another businessman, William Eckhardt, about their success. Dennis believed that anyone could learn to trade, while Eckhardt objected that Dennis had a special gift when he was able to profit from trading. This discussion created a bet.[3]

Dennis found a group of people to whom he gave the rules below and the money put into the bet. He named his students "Turtles" because of the turtle farm he saw and believed he could raise traders as fast as this turtle farm.

The result of this bet was that these trained turtles earned 175 million \$ in 5 years.

The idea of this strategy is to follow the trend. Or trend is your friend.

4.5.1 Algorithmic strategy description

This strategy contains 2 different but fundamentally similar systems. Let's call them System 1 and System 2.

Volatility - N

N is calculated as the 20-day EMA True Range (now known as ATR).

$$\text{True range} = \text{Max}(\text{High} - \text{Low}, \text{High} - \text{previous_close}, \text{Previous_close} - \text{Low})$$

$$\mathbf{N} = \frac{19 * \text{Previous}_N + \text{Truerange}}{20}$$

Position size

For a fixed SL and risk we have a simple equation

$$\text{size} * \frac{|\text{OP} - \text{SL}|}{\text{OP}} = \text{deposit} * \text{risk}$$

From this follows

$$\text{size} = \frac{\text{OP} * \text{deposit} * \text{risk}}{|\text{OP} - \text{SL}|}$$

When trading, it is important to distinguish between absolute and relative values. For each trade in this strategy, let's denote its *size* as 1 Unit

Risk

The maximum trade risk is 2%.

Entering positions

System 1

- Long open: price exceeds 20-day HIGH
- Short open: price drops below 20-day LOW

System 2

- Long open: price exceeds 55-day HIGH
- Short open: price drops below 55-day LOW

Adding to positions

Turtles add 1 Unit to an open position if the price moves $N/2$ in the direction of the original trade from the fill price of the previous trade order.

SL Location

The market movement of $1N$ represents a change of account by 1%. This means that for the risk determined above, Long SL $2N$ is placed below the entry price and Short SL $2N$ above the entry price.

When added to a position, the SL of all open trades moves $N/2$ higher. This achieves a distance of $2N$ from the last trade.

Leaving positions

System 1

- Long: price drops below 10-day LOW price level
- Short: the price rises above the price level of 10-day HIGH

System 2

- Long: price exceeds 55-day high
- Short: price drops below 55-day LOW

This system was used in the commodity market. Dennis had specific trading commodities for Turtle strategy. However, the fundamentals of these markets have changed over the years and this strategy cannot be used in its original form in these markets.

The backtest program does not allow you to test the addition of positions during an open trade and move the SL. So let's adjust the Turtle strategy for these two components. We will not add to the positions and we will leave SL in its original place. The results of the strategy are described below 5.4

5 Backtesting program

For backtesting purposes, I created a program in Python that takes candles from a given market from a csv file and analyzes them according to the rules of the strategy that the user programs in a special file. The code for this program is downloadable from gitlab [10].

Compared to existing alternatives, with this program the trader has more control over direct data and the ability to analyze the behavior of the market and individual trades deeper. However, this option also introduces different requirements for programming and data analysis capabilities from such a trader.

5.1 Strategy analysis

I describe below the results of our MA RSI, Turtle and Price-Action strategies, which we modeled earlier. The strategies were backtested on Timeframes 1W, 1D, 4H, 1H for the markets BTCUSD, ETHUSD, ETHBTC, BNBUSD on the time horizon 2017-08-18 00:00:00 – 2021-09-23 00:00:00.

Depending on the timeframe change, the size of the risk per trade should also change, however, we will omit this parameter from the analyzes below and use the risk size set out above for a specific strategy for all timeframes.

In strategy analysis, the behavior of the robot during the process may be more important than the result itself. Various tools and metrics for this can be used. I named one of them “1 Unit” in the Turtle strategy section.

Another one important to use for analysis is **RR (risk reward) gain**. At the moment when I have a fixed percentage of the deposit that risks every trade, I do not have to calculate the result in absolute terms (how much more or less money I have), but in relative terms. I will calculate this RR_gain parameter as the difference between the final and initial prices, which I divide by the difference between the initial and SL prices.

$$RR_gain = (end_price - begin_price) / (begin_price - SL_price)$$

It can be seen that if SL is reached, -1 is assigned to this trade as a result. It is easy to show that the amount of money that is get from a particular trade is obtained from the equation

$$\text{gain} = \text{deposit} * \text{risk_per_trade} * \text{RR_gain}$$

Another thing that is definitely important to measure are drawdowns. These are used to provide an overview of individual dips. In my program, I specifically measure the largest drawdown, or historically worst period in terms of account loss.

The backtesting program [10] saves the results of the analyzes in 3 files. The folder path is `Data\Outputs\{Strategy_name}\{Symbol}`

- `orders.xlsx` - records specific trades that would actually be executed
- `monthly_gain.xlsx` - records the sums of `RR_gain` in each month
- `results.xlsx` - records evaluation data, maximum drawdowns and comparison of results with and without fees. Absolute profit is recorded in multiples of the original deposit,

So now I will show the individual strategies and results that are obtained by backtesting.

5.2 MA RSI strategy

Below in Tables 5.1, 5.2, 5.3, 5.4 are for each timeframe selected parts of outputs from the `results.xlsx` file for individual symbols. At this stage, I will compare ideal trades free of charge.

5.2.1 Result analysis

Examining the results, it can be seen that the number of trades and specific results are not satisfactory, even for ideal trades without fees. When studying the `monthly_gain.xlsx` file, it can be seen that these are not a few months with big negative results, but are more or less evenly distributed.

Thus, the core of our modeled strategy has NO potential to develop further.

Table 5.1: 1W timeframe results for MA RSI strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	0	0	0	0
Number - trades	1	0	0	2
Result RR_gain	-1	0	0	-1.24
Absolute profit	-0.01	0	0	-0.01

Table 5.2: 1D timeframe results for MA RSI strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	3	1	4	4
Number - trades	3	6	1	5
Result RR_gain	11.88	-3.5	9.37	16.54
Max drawdown (RR)	1.52	3.23	0.4	1.24
Absolute profit	0.12	-0.03	0.1	0.16

Table 5.3: 4H timeframe results for MA RSI strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	16	15	20	13
Number - trades	44	52	33	45
Result RR_gain	1.1	15.22	39.97	-14.72
Max drawdown (RR)	9.89	142	4.29	14.40
Absolute profit	0	0.15	0.44	-0.14

Table 5.4: 1H timeframe results for MA RSI strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	68	57	72	68
Number - trades	207	184	142	153
Result RR_gain	-11.14	10.23	18.93	17.56
Max drawdown (RR)	44.64	44.63	18.08	16.71
Absolute profit	-0.14	0	0.17	0.16

Table 5.5: 1W timeframe results for Price action strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	7	4	6	6
Number - trades	5	4	5	6
Result RR_gain	2	0	1	0
Absolute profit	0.02	0	0.01	0

Table 5.6: 1D timeframe results for Price action strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	47	33	35	39
Number - trades	60	52	43	47
Result RR_gain	-13	-19	-8	-8
Max drawdown (RR)	18	24	12	11
Absolute profit	0.13	-0.18	-0.08	-0.08

5.3 Price action strategy

In Tables 5.5, 5.6, 5.7, 5.8 are selected for each timeframe selected selected parts of outputs from the file `results.xlsx` for individual timeframes and symbols. At this stage, I will compare ideal trades free of charge.

Table 5.7: 4H timeframe results for Price action strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	301	279	282	283
Number - trades	378	319	314	319
Result RR_gain	-77	-40	-32	-36
Max drawdown (RR)	77	48	55	39
Absolute profit	-0.55	-0.35	-0.3	-0.32

Table 5.8: 1H timeframe results for Price action strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	1228	57	1174	1038
Number - trades	1221	184	1343	1130
Result RR_gain	7	10.23	-169	-92
Max drawdown (RR)	98	54	193	118
Absolute profit	-0.05	0	-0.84	-0.64

Table 5.9: 1W timeframe results for System 1 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	3	1	3	2
Number - trades	4	4	5	3
Result RR_gain	9.45	-1.76	-2.66	0.62
Absolute profit	0.09	-0.02	-0.03	0.01

5.3.1 Result Analysis

The conclusion is very similar to the previous strategy. Examining the results, it can be seen that the number of trades and specific results are not satisfactory, even with ideal trades without fees. When studying the `monthly_gain.xlsx` file, it can again be found that these are not a few months with big negative results, but are more or less evenly distributed.

The core of this modeled strategy therefore has NO potential to develop further.

5.4 Turtle strategy

Below in tables 5.9, 5.10, 5.11, 5.12, 5.13, 5.14, 5.15, 5.16 are the outputs for System 1 and System 2, as we defined above. At this stage, we will compare ideal trades free of charge.

Table 5.10: 1D timeframe results for System 1 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	18	22	22	18
Number - trades	21	13	22	24
Result RR_gain	94.24	67.16	37.67	101.62
Max drawdown (RR)	3.59	2.65	2.71	6.51
Absolute profit	1.21	0.85	0.44	1.38

Table 5.11: 4H imeframe results for System 1 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	97	102	118	104
Number - trades	165	167	153	161
Result RR_gain	133.07	149.02	142.8	134.94
Max drawdown (RR)	7.28	11.47	13.51	12.37
Absolute profit	2.43	2.97	2.89	2.12

Table 5.12: 1H timeframe results for System 1 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	389	415	369	394
Number - trades	719	668	682	702
Result RR_gain	228.38	315.79	293.05	241.95
Max drawdown (RR)	19.64	14.28	37.34	16.54
Absolute profit	6.71	18.13	13.87	7.96

Table 5.13: 1W timeframe results for System 2 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	0	0	0	0
Number - trades	3	2	3	2
Result RR_gain	-2.22	-1.16	-2.18	-1.47
Absolute profit	-0.02	-0.01	-0.02	-0.01

5.4.1 System 1

5.4.2 Analyzing System 1 Results

Compared to the other two strategies mentioned above, interesting evaluations can be seen at first sight. However, there is really a very small number of trades on the 1W timeframe that can be more affected by the market fundamentals at a given moment than the strategy itself. The situation is more interesting at 1D timeframe. However, the appreciation of +359% and 94.24 RR has one catch. October 2020 gained +60 RR, which significantly impairs the continuous ability of the strategy to generate profit.

At lower timeframes, the situation with absolute results is even more significant. This shows how important it is to take data in the context of continuous performance, not just absolute numbers. E.g. after taking into account the fees and inaccuracies of the Market entry, we get from the absolute profit of +671% to about 90%. In other markets, this difference is even more pronounced.

According to current requirements and metrics, this strategy is successful. Further steps can lead to an analysis of extreme periods, both in terms of losses and profit.

5.4.3 System 2

5.4.4 Analyzing System 2 Results

Results similar to System 1 can be seen. The 1W timeframe is again unusable due to the small number of trades and their relevance to market fundamentals. Compared to System 1, it can be seen that there are about 1/3 less trades, but it cannot be said that the robot has a better value in absolute numbers.

Table 5.14: 1D timeframe results for System 2 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	9	12	11	9
Number - trades	10	7	10	14
Result RR_gain	64.02	59.91	18.69	99.86
Max drawdown (RR)	3.38	1.62	2	5
Absolute profit	0.76	0.69	0.2	1.38

Table 5.15: 4H timeframe results for System 2 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	44	50	51	40
Number - trades	83	85	76	85
Result RR_gain	137.93	109.95	151.13	128.66
Max drawdown (RR)	8.41	9.18	10.38	14.31
Absolute profit	2.47	1.79	2.91	1.89

Table 5.16: 1H timeframe results for System 2 Turtle strategy

	BTCUSD	ETHUSD	ETHBTC	BNBBTC
Number + trades	191	192	203	194
Number - trades	361	371	368	517.77
Result RR_gain	213.77	265.93	265.26	235.58
Max drawdown (RR)	16.82	14.17	19.19	13.40
Absolute profit	5.96	10.61	10.31	7.69

At this point, it is important to take into account the fees and non-ideality of opening stores. The numbers here clearly speak in favor for System 2.

According to current requirements and metrics, this strategy is successful. Further steps can lead to an analysis of extreme periods, both in terms of losses and profit. A deeper comparison between Systems 1 and 2 and a reflection on their possible combination.

5.5 Evaluation of backtesting strategies

Using a programmed backtesting program, I was able to effectively filter out unusable strategies so that I could focus on the profitable ones in the next steps. The individual analyzes show the shortcomings of the programmed application in comparison with the platforms mentioned in the introduction, which can be used for backtesting purposes.

Such shortcomings may be the absence of a display strategy, such as inserting trades into a graphic chart and analyzing whether they are evenly distributed over time or, conversely, appear larger clusters of time, or account appreciation over time. This user-friendly access parameter can be found, for example, on the above-mentioned Ninja-Trader. Other shortcomings may be limited in terms of creating orders. In the current version of the program, for example, Trailing trades or Limit and Stop market orders for opening new trades are not combined by default.

The next steps in working with this application may be to program additional indicators, improve the documentation, put into operation Limit and Stop market inputs and add other types of analysis usable for working with strategies.

However, the real work on this application will significantly depend on the needs of current projects and strategies that they devise and will need to test.

6 Conclusion

The goal was to create a backtest program for trading strategies, over which the user has more control when working with data than if he used methods and platforms already created.

In this work, I described a few problems that the trader or the backtester of strategies can meet. They got acquainted with algorithmic proposals of various strategies, which included a brief insight into the world of indicators, price action and game theory.

I evaluated the individual approaches to creating strategies according to the fundamentals and the market principle both during and at the end of the backtesting of the strategies themselves, on which I showed the imperfections of the analyzes created using the new backtesting program. I specified these imperfections and recommendations at the end of this work.

The program itself can be considered a success to the specified parameters, which will complement the offer of programs designed to analyze strategies. Through the size of the market, the requirements and the specificity of the capabilities that are placed on users, there is no ambition for the wider dissemination of this program.

Bibliography

- [1] URL: https://a.c-dn.net/c/content/dam/publicsites/igcom/uk/images/ContentImage/How-to-read-forex-charts-candlestick-Green-and-red-HLOC-bars_LRG.png (visited on 07/05/2022).
- [2] URL: <https://a.c-dn.net/c/content/dam/publicsites/igcom/uk/images/ContentImage/BULLISH%20BEARISH%20CANDLESTICK-150620.jpg> (visited on 01/05/2022).
- [3] URL: <https://oxfordstrat.com/coasdfASD32/uploads/2016/01/turtle-rules.pdf> (visited on 09/09/2021).
- [4] URL: <https://www.bitmex.com/app/fees> (visited on 10/05/2022).
- [5] 3Commas. URL: <https://3commas.io> (visited on 01/05/2022).
- [6] Vineet Kumar Aarti Goswami Pramod Borasi. *Cryptocurrency Market Size, Share and Analysis: Forecast - 2030*. URL: <https://www.alliedmarketresearch.com/crypto-currency-market> (visited on 03/05/2022).
- [7] *Bitcoin vs The Biggest Companies And Assets In The World by Market Cap*. URL: <https://coinmarketcap.com/largest-companies/>.
- [8] Brian J. Bloch. *How to Develop a "Trading Brain"*. URL: <https://www.investopedia.com/articles/basics/13/how-to-develop-trading-brain.asp> (visited on 12/03/2022).
- [9] Ondřej Buček. URL: https://gitlab.fi.muni.cz/xbucek3/bc_thesis_scheduler.
- [10] Ondřej Buček. URL: https://gitlab.fi.muni.cz/xbucek3/bc_thesis_backtesting.
- [11] Theresa W. Carey. *20 years old Turtles strategy still work!!* URL: <https://www.tradingview.com/script/Q1023zJP-20-years-old-turtles-strategy-still-work/> (visited on 10/12/2021).
- [12] Theresa W. Carey. *Top Technical Analysis Tools for Traders*. URL: <https://www.investopedia.com/articles/active-trading/121014/best-technical-analysis-trading-software.asp>.
- [13] James Chen. *Technical Indicator Definition*. URL: <https://www.investopedia.com/terms/t/technicalindicator.asp> (visited on 05/05/2022).

BIBLIOGRAPHY

- [14] Jaroslav Chudáček. *Obchodování bitcoinů jednoduchou matematickou metodou*. URL: <https://btctip.cz/obchodovani-bitcoinu-jednoduchou-matematickou-metodou/> (visited on 09/08/2021).
- [15] Jason Fernando. *Moving Average (MA)*. URL: <https://www.investopedia.com/terms/m/movingaverage.asp> (visited on 06/05/2022).
- [16] Jason Fernando. *Relative Strength Index (RSI)*. URL: <https://www.investopedia.com/terms/r/rsi.asp> (visited on 06/05/2022).
- [17] Jean Folger. *Top 10 Rules For Successful Trading*. URL: <https://www.investopedia.com/articles/trading/10/top-ten-rules-for-trading.asp> (visited on 03/03/2022).
- [18] Adam Hayes. *Martingale System*. URL: <https://www.investopedia.com/terms/m/martingalesystem.asp> (visited on 06/05/2022).
- [19] "How to Write a Good README File for Your GitHub Project". In: (). URL: <https://www.freecodecamp.org/news/how-to-write-a-good-readme-file/> (visited on 10/05/2022).
- [20] Investopedia. URL: <https://www.tradingview.com/about/> (visited on 09/05/2022).
- [21] *Make a README*. URL: <https://www.makeareadme.com> (visited on 09/05/2022).
- [22] *MetaTrader5*. URL: <https://pypi.org/project/MetaTrader5/> (visited on 03/04/2022).
- [23] *MQL4 Reference*. URL: <https://docs.mql4.com>.
- [24] *MQL5 Reference*. URL: <https://www.mql5.com/en/docs>.
- [25] Prashant Raut. *Different Types of Price Charts for Trading: Stock Price Chart*. URL: <https://www.tradingfuel.com/types-of-price-charts-for-trading/> (visited on 04/04/2022).
- [26] *TradingView plans*. URL: <https://www.tradingview.com/gopro/#compare>.
- [27] TrendSpider. URL: <https://www.trendspider.com> (visited on 01/05/2022).
- [28] *Types of Orders*. URL: <https://www.investor.gov/introduction-investing/investing-basics/how-stock-markets-work/types-orders> (visited on 01/03/2022).
- [29] *Welcome to Pine Script*. URL: <https://www.tradingview.com/pine-script-docs/en/v5/Introduction.html>.